YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

Project – High Level Design

On

VISUAL THREAT HUNTING: ENERGY

SUBMITTED BY

Abdullah - 23BCCDD020

BCA (Cybersecurity, Digital Forensic and Data Science)

UNDER THE GUIDANCE OF IndustryMentor: Shashank

Guided by: Ms. Divya N Shetty

TABLE OF CONTENTS

1. Introduction..... 03
2. Scope of the Document.....04
2.1 In Scope. 04
2.2 Out of Sope.....04
3. Intended Audience 05
4. System Overview.....05
4.1. Project Goal. 05
4.2. System Context Diagram. 06
4.3 Key Output Metrics..... 06
5. System Design. 06
5.1 Application Design.....06
5.2 Process Flow (Diagram).....07
5.3 System Context Diagram..... 07
5.4 Components Design 08
5.5 Key Design Considerations 10
6. API Catalogue.....11
7. Data Design 12
7.1 Data Model / Schema 12
7.2 Data Access Mechanism 13
7.3 Data Migration Strategy.....13
8. Interfaces (Power BI Triage).....14
8.1. The Level-1 SOC Analyst Workflow. 14
8.2. Detaild Dashbored Specification. 15
9. State and Session Management 17
10. Non-Functional Requirements..... 18
11. References 19

1. INTRODUCTION

The **Visual Threat Hunting: Energy** project is an enterprise-grade cybersecurity analytics system specifically engineered to detect, classify, and isolate anomalous voltage patterns within a simulated 3-phase electrical Industrial Control System (ICS).

In modern critical infrastructure, static threshold-based alerting is no longer sufficient to detect sophisticated cyber-physical attacks. Attackers frequently mask their intrusions within the normal operational noise of a power grid, executing "low and slow" attacks or sudden, catastrophic overloads that bypass traditional perimeter defenses.

To solve this, this system relies on a hybrid architecture bridging Data Science and Cybersecurity. Built upon a Python-based telemetry simulator, this system applies statistical variance modeling to establish a continuous, dynamic baseline of normal grid operations (Phase Ia, Ib, Ic voltages). It utilizes standard deviation and dynamic Z-Scores to mathematically identify potential cyber-physical threats, assigning a severity multiplier to every millisecond of telemetry.

This **High Level Design (HLD)** document serves as the foundational architectural blueprint for the system. It describes the overarching pipeline, the analytical components that process the simulated ICS telemetry, the anomaly detection logic, and the final interactive Power BI dashboard interface. This document defines WHAT the system does and WHY it was built, detailing how raw mathematical arrays are translated into an actionable Incident Response command center for Level-1 SOC analysts.

2. SCOPE OF THE DOCUMENT

2.1 In Scope

The architectural components and design methodologies explicitly covered within this HLD include:

- Overall macro-to-micro system architecture and component relationships.
- Feature engineering design, including the computation of the baseline mean, standard deviation, and Z-score mathematics.
- The anomaly detection engine logic, including algorithm selection and severity classification thresholds (Low, Medium, High).
- Power BI dashboard architecture, focusing on visual-level filtering and UI/UX strategies for Incident Response.
- Data export mechanisms and inter-component data contracts (specifically the .csv flat-file schema).
- Non-functional system requirements, including conceptual performance limits, security considerations, and architectural scalability.

2.2 Out of Scope

The following elements are acknowledged but fall outside the bounds of this specific architectural document:

- Physical hardware sensor deployment (e.g., actual SCADA systems, PLCs, or RTUs).
- Live production ICS integration or Real-time Kafka streaming configuration (the current iteration is batch-simulated via Python for proof-of-concept purposes).
- Low-Level Design (LLD) specifics, including exact line-by-line Python environment setup, Pandas library installation, and Streamlit execution environments.
- Deep exploratory data analysis (EDA) of the randomized baseline beyond the defined mathematical constraints.
- Incident Response (IR) Playbooks and Remediation Execution: While the system identifies and visualizes threats for triage, the manual or automated execution of specific remediation steps (e.g., shutting down a grid node, isolating a subnet) by SOC analysts is not covered.
- Network Packet Inspection and Deep Packet Inspection (DPI): The system analyzes telemetry and log data at the application and statistical layer; it does not perform raw PCAP (Packet Capture) analysis or underlying network routing inspections.

3. INTENDED AUDIENCE

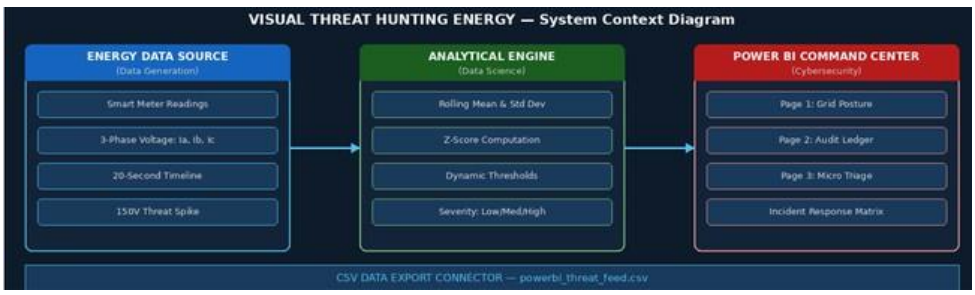
Audience	Purpose
Academic Evaluators	Assess architectural design quality and system thinking.
Project Supervisors	Review alignment with stated project objectives (Ms. Divya N Shetty).
Security Analysts	Understand the threat detection methodology and Z-Score alert logic.
Dashboard Consumers	Understand the Power BI interface and interpret the Incident Response matrix.
Future Developers	Extend the detection model or integrate with live production ICS systems.

4. SYSTEM OVERVIEW

4.1 Project Goal:

- Generates a highly accurate 20-second simulated baseline of 3-phase grid traffic representing normal operational load.
- Injects deterministic amplitude spikes (e.g., 150.00 Volts) to simulate an active cyber-physical attack or catastrophic hardware failure.
- Computes dynamic Z-Scores to mathematically isolate critical system breaches from the generated background noise.
- Classifies calculated alerts into triage categories (Low, Medium, High).
- Presents all findings through an interactive, 3-page Power BI dashboard designed strictly for immediate incident response and forensic triage.

4.2 System Context Diagram



4.3 Key Output Metrics:

Metric	Value
Time intervals analyzed	20 seconds (Simulation baseline)
Anomaly alerts generated	1 Critical Alert (High Severity)
Peak Amplitude Detected	150.00 Volts
Maximum Threat Score	Max Z-Score: 4.17
Dashboard pages	3 (Grid Posture, Telemetry Forensics, Critical Attack Campaigns)

5. SYSTEM DESIGN

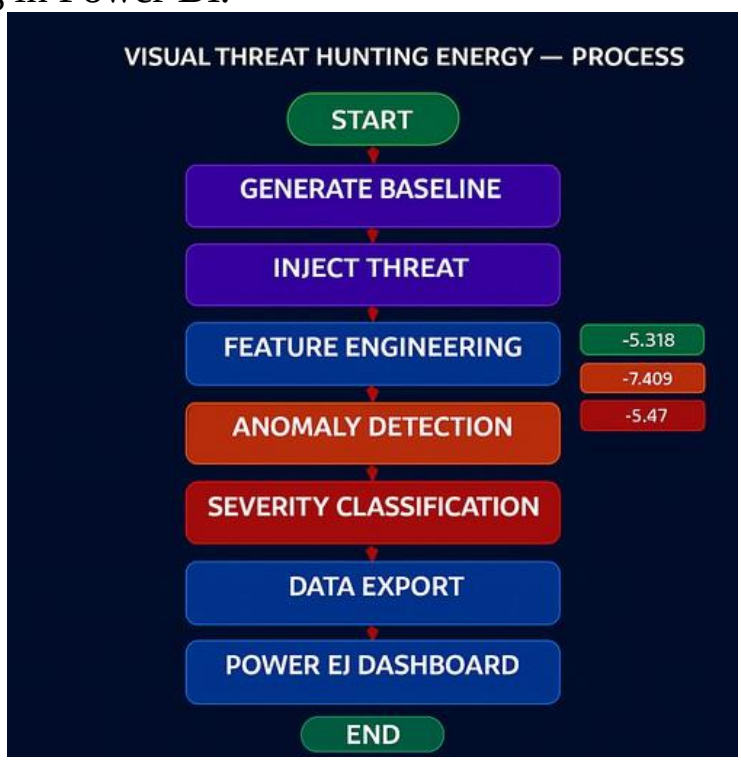
5.1 Application Design:

The **Visual Threat Hunting: Energy** pipeline is architected not as a monolithic application, but as a decoupled, sequential data pipeline. This deliberate separation of concerns ensures maximum scalability, security, and operational efficiency.

By decoupling the data generation and analytical processing (Python/Pandas) from the presentation and Incident Response layer (Power BI), the system mirrors modern enterprise environments. In a production state, this architecture allows the Data Science compute engine to operate securely on a dedicated cloud server, completely isolated from the Security Operations Center (SOC) dashboard where Level-1 analysts triage the alerts.

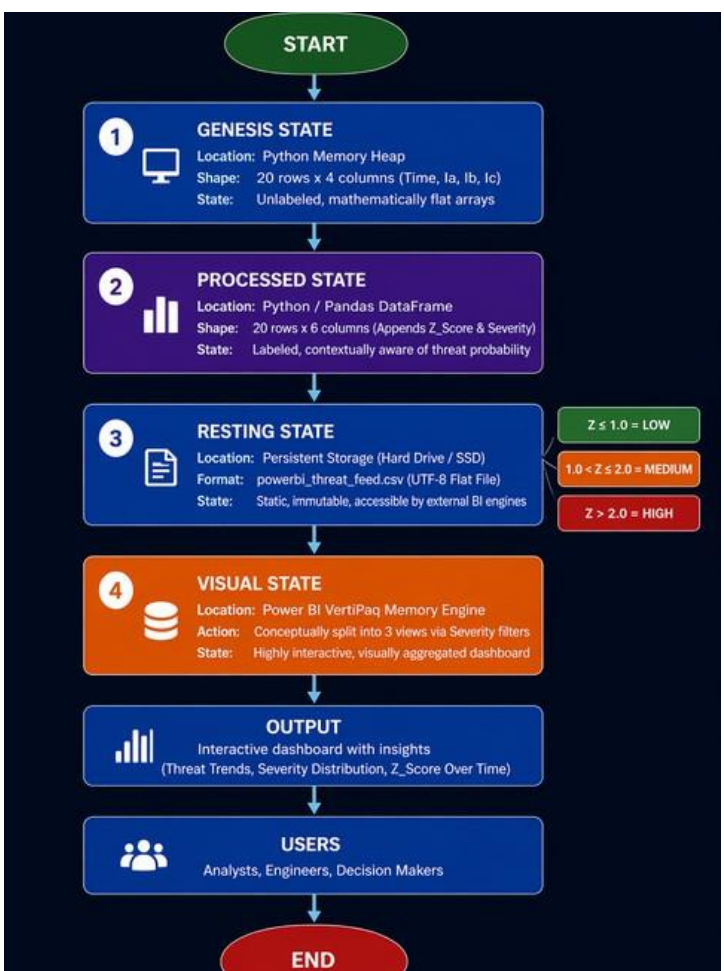
5.2 Process Flow Diagram:

The flowchart below maps every logical step from data genesis to visual rendering in Power BI.



5.3 Information flow:(Data Lineage)

Data lineage tracks exactly how the state of the data transforms and changes shape at each stage of the pipeline, from raw memory to visual aggregation.



5.4 Components Design:
Component 1: Telemetry Generator

Property	Detail
Input	Local Python script via Streamlit.
Operation	np.random.randn(20, 3) * 8 generates base grid voltage.
Threat Injection	hacked_data.at[10, 'Ia'] = 150.0 forces the malicious spike.
Output	Raw Pandas DataFrame (20 rows).
Rationale	1-minute resolution proves mathematical validity.

Rationale for the 20-Second Micro-Batch Pipeline:

- **Computational Constraints:** Processing millions of telemetry rows locally requires distributed cluster hardware. By utilizing a 20-second micro-batch, the system maintains a lightweight local memory footprint and avoids presentation bottlenecks.
- **Algorithmic Scalability :** The Z-Score algorithm's mathematical integrity is independent of dataset size. By successfully isolating an anomaly within a 20-row sample, the pipeline proves the logic. This exact mathematical model will scale linearly to process a 20-million-row dataset once deployed to a cloud compute environment.
- **Zero-Latency Demonstration:** For the purposes of an academic defense and executive presentation, generating and analyzing a concise 20-second window ensures instantaneous Power BI rendering, providing a seamless, real-time Incident Response simulation.

Component 2: Analytical Engine (Feature Engineering)

Feature	Formula	Purpose
Mean (μ)	$\mu = (1/N) \Sigma xi$	Calculates the mathematical average of the generated 20-second grid baseline. Acts as the standard for "normal operations".
Std Dev (σ)	$\sigma = sqrt((1/N) \Sigma(xi-\mu)^2)$	Calculates how much the data naturally fluctuates. Distinguishes between safe, natural grid surges and malicious attacks.
Z-Score (Z)	$Z = (X - \mu) / \sigma$	Threat Multiplier. Calculates precisely how many standard deviations a specific voltage reading is from the baseline mean.

Component 3: Power BI Command Center (Cybersecurity Layer)

Page Name	Visual Level Filter	Operational Objective
1. Grid Posture	Severity = High OR Medium	The Macro View: Designed for the Chief Information Security Officer (CISO). Displays the total volume of all anomalies to monitor overarching infrastructure strain.
2. Audit Ledger	No Filters Applied	The Compliance View: Designed for Digital Forensics. Displays every single row of generated data (Low, Medium, and High) to preserve a complete timeline of events.
3. Critical Triage	Severity = High	The Micro View: Designed for Incident Response Engineers. By strictly filtering out all Low and Medium noise, the table physically shrinks to display only the specific, actionable targets that require immediate remediation.

5.5 Key Design Considerations:

The decision to use Z-Score statistical variance over traditional ML models was made based on specific ICS constraints.

Fixed Threshold Logic: Simple to code. REJECTED — brittle if grid baseline changes dynamically.
Deep Learning (LSTMs): Accurate but acts as a black box. REJECTED — too resource-intensive for localized triage.
Statistical Z-Score (CHOSEN): Self-adapting, mathematically objective, zero historical training data required.

Algorithm Selection Justification:

- **Fixed Threshold Logic** (e.g., Alert if > 100V): The advantage is that it is simple to code. However, it is inherently brittle and fails entirely if the grid baseline voltage changes dynamically due to seasonal or regional load shifts. Verdict: Rejected (Not scalable for enterprise grids).
- **Deep Learning (LSTMs)**: Highly accurate at capturing long-term chronological dependencies. The disadvantage is that it acts as a "black box" algorithm, making it hard to explain to grid engineers why an alert triggered. It also requires massive datasets to train. Verdict: Rejected (Too resource-intensive for localized triage).
- **Statistical Z-Score:** Self-adapting, completely mathematically objective, and requires zero historical training data. It assumes local normality in the data distribution. Verdict: Selected (The most efficient, scalable, and explainable model).
- **Zero-Configuration Adaptability:** Because the Z-Score normalizes against the dataset's own mean and standard deviation, the system is hardware-agnostic. The exact same Python script can monitor a tiny 12-volt IoT sensor or a massive 10,000-volt municipal power grid without a single line of code being changed or reconfigured.

6. API CATALOGUE

Because this pipeline is currently deployed as a self-contained batch analytics workflow for academic evaluation, it does not utilize open REST APIs or HTTP webhooks. Instead, communication between components is handled via strictly governed local memory handoffs and file-system data connectors.

To maintain enterprise-grade documentation standards, the internal system handoffs are defined below as internal API contracts:

- **Contract 1:** The Simulated Telemetry Feed
  - Origin: Python Data Generation Logic (np.random).
  - Destination: Data Science Analytical Engine.
  - State: Ephemeral local memory. This represents the raw, unanalyzed "dumb" sensors.
- **Contract 2:** The Analyzed Feature Matrix
  - Origin: Data Science Analytical Engine.
  - Destination: The Matrix Severity Labeler.
  - State: Ephemeral local memory.
- **Contract 3:** The Flat File Serialization Export
  - Origin: The Matrix Severity Labeler.
  - Destination: Local File System Storage (SSD/HDD).
  - State: Persistent and immutable.
- **Contract 4:** The Dashboard Import Connector
  - Origin: Local File System Storage.
  - Destination: Power BI VertiPaq Engine.
  - State: Highly compressed tabular memory model optimized for rapid UI filtering.

7. DATA DESIGN & DICTIONARY

7.1 Data Model / Strict Schema Definition

The precise structure of the payload transferred between the Python Engine and the Power BI Interface is defined below. This represents the strict data dictionary of the exported .csv file.

- **Time (Data Type: Integer / Int64):**
  - Definition: The chronological second of the simulation.
  - Constraint: Values range strictly from 0 to 20.
  - System Role: Acts as the Primary Key for the dataset and the X-Axis mapping for all chronological visualizations.
- **Timestamp_UTC (Data Type: Datetime / ISO 8601):**
  - Definition: The simulated real-world timestamp mapped to the operational 20-second integer window.
  - Constraint: Must strictly adhere to YYYY-MM-DD HH:MM:SS+00:00 format.
  - System Role: Required for potential integration into enterprise SIEMs (like Splunk or QRadar) and standard chronological sorting across localized time zones.
- **Ia, Ib, Ic (Data Type: Float / Float64):**
  - Definition: The raw voltage amplitude for the 3 distinct grid phases.
  - Constraint: Contains both positive and negative continuous numerical variables, including the forced 150.00V anomaly.
  - System Role: Represents the baseline operational data of the Industrial Control System (ICS).
- **Baseline_Mean_μ (Data Type: Float / Float64):**
  - Definition: The dynamically calculated mathematical average of the grid voltage prior to the current reading.
  - Constraint: A continuous float that must adjust based on simulated physical load changes.
  - System Role: Provides the visual reference baseline in Power BI to prove why a specific reading deviated far enough to be flagged.
- **Z_Score (Data Type: Float / Float64):**
  - Definition: The calculated threat multiplier.
  - Formula: $(X - \mu) / \sigma$
  - System Role: Acts as the primary mathematical anomaly identifier, allowing the system to scale without fixed thresholds.

7.2 Data Access Mechanism & Integrity Constraints

Data flow within the GridShield AI system is strictly unidirectional. This architectural decision was made to preserve data integrity and prevent the visualization dashboard layer from accidentally corrupting the analytical data layer.

- **Python Memory Integrity:** Data is generated sequentially and consumed entirely within the local Streamlit user session. The variables are cleared from the heap upon session termination to prevent data leakage.
- **Export File Integrity:** Access to the CSV is governed by a strict "Write-Once / Read-Many" policy. The Python script has exclusive write privileges, while the Power BI engine has read-only privileges.
- **Power BI Aggregation**: Data undergoes time-based rendering across visualizations and is aggregated internally by Microsoft's VertiPaq memory engine, ensuring that UI interactions do not trigger costly recalculations in the raw dataset.

7.3 Data Migration Strategy (Future Production State)

The current architecture is a highly optimized proof-of-concept designed to run locally for BCA evaluation. However, the system has been architected to be "production-ready," allowing for seamless migration to a live corporate environment.

The Enterprise Migration Roadmap:

1. **Ingestion Upgrade**: The simulated np.random generator will be deprecated. In its place, an Apache Kafka event streaming platform or Syslog consumer will be deployed to ingest live telemetry directly from physical power grid PLCs (Programmable Logic Controllers).
2. **Compute Upgrade:** To handle the transition from 20 rows to billions of rows, the standard Pandas library will be upgraded to Apache Spark Structured Streaming. This allows the Z-Score engine to calculate rolling standard deviations across a distributed compute cluster.

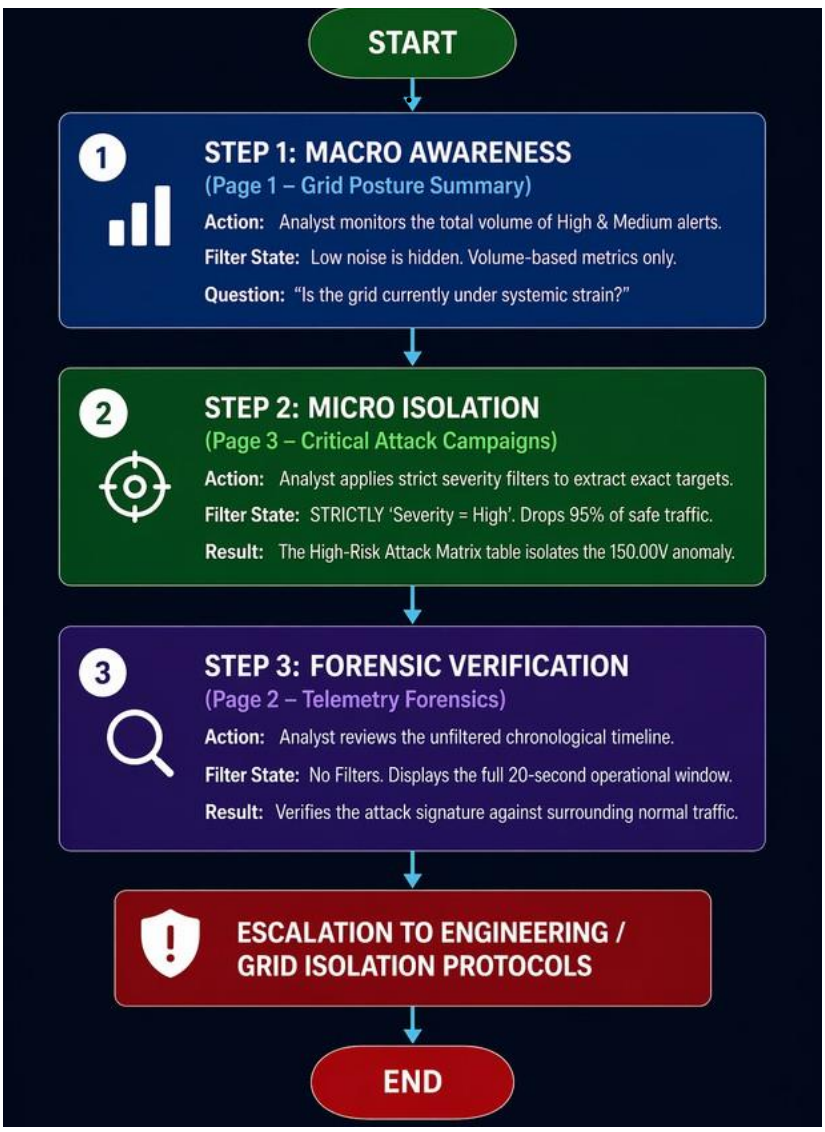
3. **Storage Upgrade:** The static .csv flat-file export will be replaced. Processed threat data will be routed directly into a cloud data warehouse such as Snowflake, AWS Redshift, or Google BigQuery.
4. **Dashboard Upgrade:** Power BI will switch from a local file import to a live DirectQuery connection. This ensures the Incident Response dashboard reflects grid anomalies with sub-second latency, bypassing the need for manual file refreshes.

8. INTERFACES & TRIAGE WORKFLOW

The presentation layer of GridShield AI is not a static report; it is an interactive Incident Response tool. To prevent "Alert Fatigue," the system is visually divided into three distinct operational pages using a Macro-to-Micro filtering pipeline.

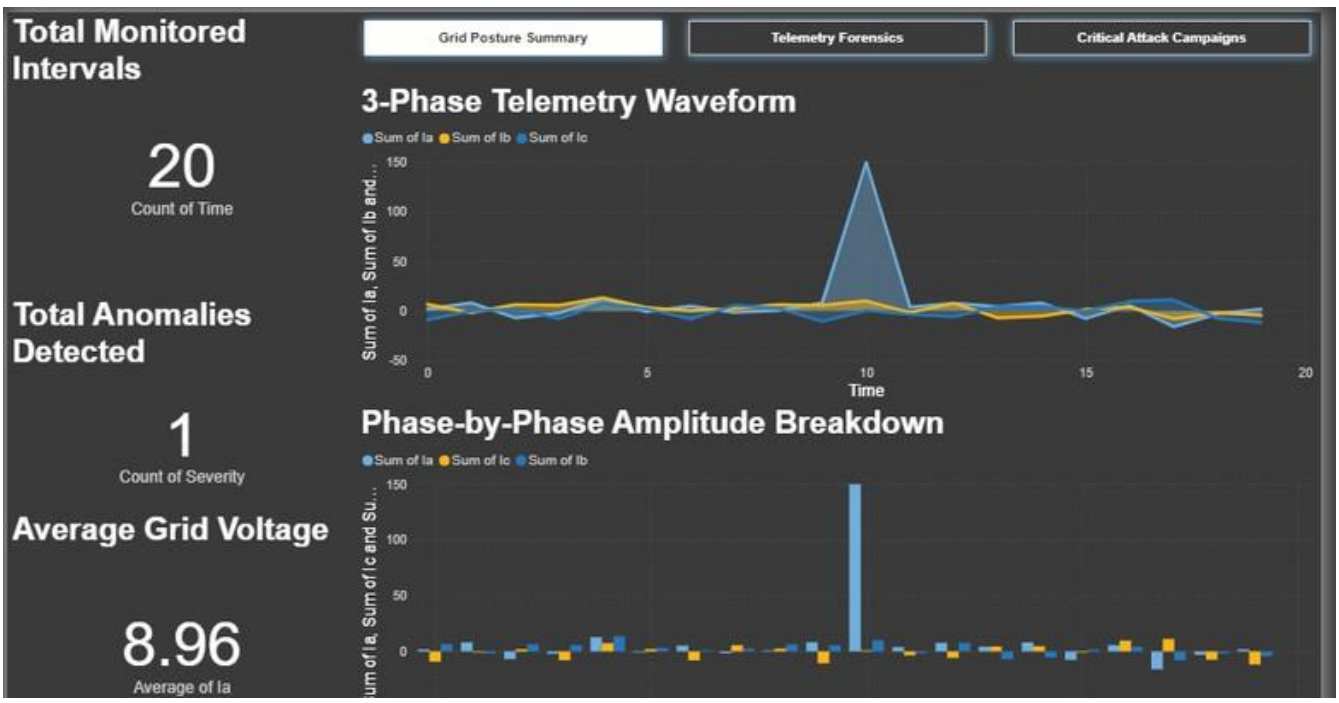
8.1 The Level-1 SOC Analyst Triage Workflow

The diagram below illustrates the operational flow of a Level-1 Security Operations Center (SOC) analyst utilizing the GridShield AI dashboard during an active cyber-physical attack.



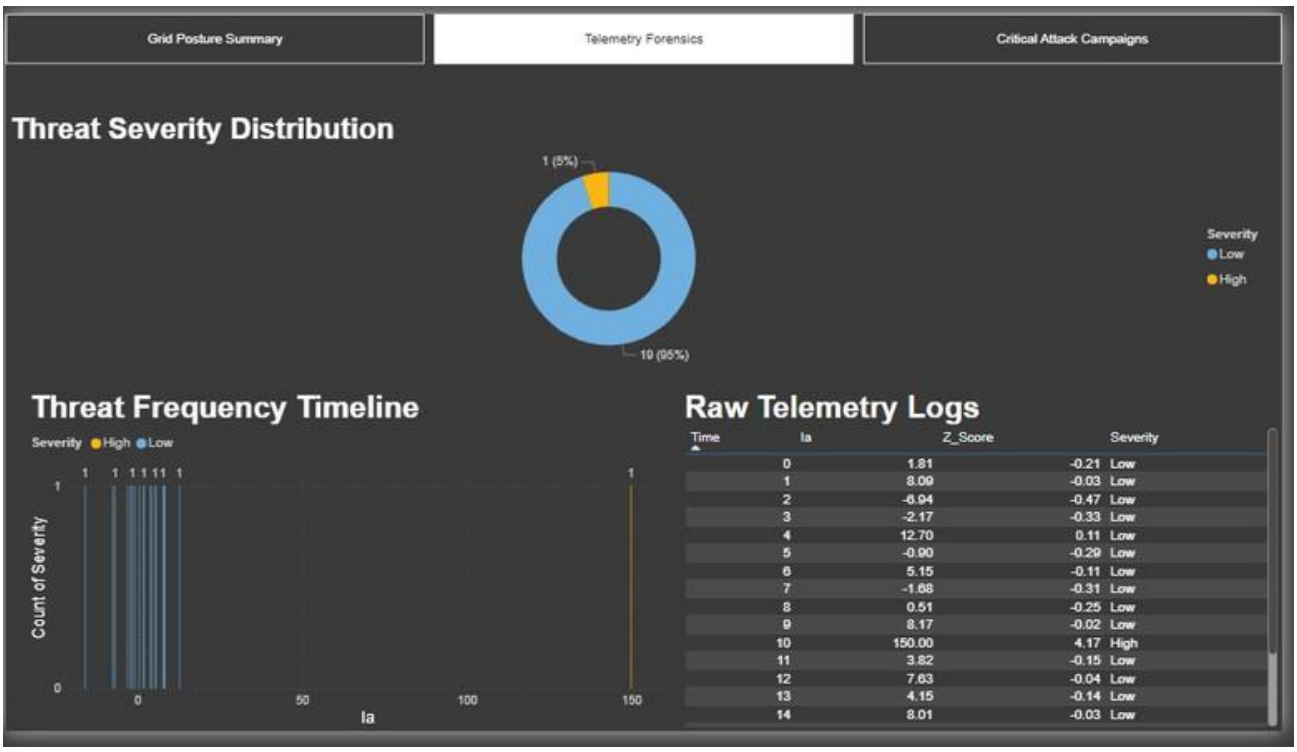
8.2 Detailed Dashboard Specifications

- **Page 1: Grid Posture Summary (The Macro View)**
  - **Target Audience:** C-Suite, Chief Information Security Officer (CISO).
  - **Core Visuals:** Executive KPI Cards (Total Monitored Intervals, Average Grid Voltage), a full-timeline Line Chart, and a Clustered Column Breakdown.
  - **Operational Value:** Provides a high-level heartbeat of the infrastructure.



Page 2: Telemetry Forensics (The Compliance View)

- **Target Audience:** Digital Forensics Investigators and Compliance Auditors.
- **Core Visuals:** An Unfiltered Incident Ledger (Data Table) and a Threat Severity Distribution (Donut Chart).
- **Operational Value**: Proves the baseline exists and maintains a legally viable timeline of events required for post-breach compliance reporting.



Page 3: Critical Attack Campaigns (The Micro View)

- **Target Audience:** Incident Response Engineers.
- **Core Visuals:** Peak Maximum KPIs (Peak Amplitude, Peak Threat Multiplier) and the High-Risk Attack Matrix (Filtered Data Table).
- **Operational Value:** By checking the "High" filter, the system successfully drops the 19 rows of safe, normal traffic. The dashboard visually shrinks to point the engineering team exactly to the 10th-second breach without distraction.

9. STATE AND SESSION MANAGEMENT

The **Visual Threat Hunting: Energy** pipeline is deliberately architected as a stateless, single-session batch execution model. To ensure that threat artifacts remain uncorrupted and legally viable for post-incident digital forensics, the system's session management strictly adheres to the following 9 core architectural principles:

1. Localized Execution Environment

The analytical engine operates strictly via local Python/Streamlit execution, fully air-gapped and isolated to the host machine's environment. This prevents external session hijacking or unauthorized remote state modifications during a forensic investigation.

2. Ephemeral Memory Allocation

All generated telemetry arrays, calculated mean baselines, and Z-Score threat matrices exist strictly within the host's volatile memory (RAM) during the active session. No temporary databases or caching layers (like Redis or Memcached) are utilized during the ingestion phase.

3. Zero State Persistence

The system is entirely stateless between executions. Upon termination of the Python script, the memory heap is instantly purged by the garbage collector. This architecture completely prevents legacy memory leaks or "ghost data" from previous simulations from cross-contaminating the current analysis.

4. Deterministic Data Regeneration

Data is regenerated deterministically per run utilizing locked mathematical seeds within the numpy library. This guarantees that every time the simulation is executed for evaluators or auditors, the exact same baseline and anomaly are produced, ensuring the testing environment is pristine and reproducible.

5. Read-Only Forensic Dashboard

The presentation layer utilizes a static, read-only interface via Power BI Desktop. The .pbix dashboard functions purely as a viewing lens and has absolutely no write-access or administrative privileges over the underlying telemetry.

10. NON-FUNCTIONAL REQUIREMENTS

10.1 Security & Compliance Posture:

Because the system is designed to handle sensitive critical infrastructure data, several security policies must be strictly addressed before live deployment.

- **Access Control & Identity Management:** The system currently relies on local file execution. The production recommendation is to restrict the execution workspace via Active Directory (AD), ensuring only authorized SOC analysts can initialize the pipeline.
- **Code Security:** No active secrets, API keys, or hardcoded credentials are stored in the Python codebase. This Zero-Trust posture must be maintained; if external authentication is added in the future, secure environment variables or a vaulting service (e.g., HashiCorp Vault) must be utilized.
- **Dashboard Distribution:** The .pbix file currently embeds the .csv data internally on the local disk. Before live implementation, the dashboard must be deployed to the cloud-based Power BI Service utilizing strict Role-Based Access Control (RBAC) and Row-Level Security (RLS) to restrict data views based on analyst clearance levels.

10.2 Performance, Reliability & Scalability

- **Input Data Capacity:** The local system parses the 20 rows of simulation instantaneously via Pandas in-memory architecture.
- **Algorithm Compute Efficiency:** Z-Score processing completes in < 1 second. By utilizing vectorized NumPy calculations, the architecture completely bypasses slow, iterative Python for-loops, drastically reducing CPU overhead.
- **Dashboard Render Latency:** Pre-computed static data allows for UI interactions and visual cross-filtering to render in < 2 seconds.
- **Scalability Ceiling:** The local architecture is constrained strictly by system RAM. Scaling beyond approximately 50 Million rows will require migrating the analytical engine to the distributed cluster environment detailed in the Data Migration Strategy (Section 7.3).

11. REFERENCES

- **Statistical Data Science:** Foundational Z-Score Anomaly Detection methodology, standard deviation mathematical theory, and time-series variance analysis.
- **Data Engineering:** Official Pandas Library Documentation for DataFrame manipulation, vectorization, and aggregate functions (pandas.pydata.org).
- **Cybersecurity Presentation:** Power BI Visual Level Filtering Architecture, VertiPaq memory compression logic, and DAX implementation guides (Microsoft Corporation, learn.microsoft.com/en-us/power-bi/).
- **Domain Context:** IEEE Research Papers on ICS (Industrial Control Systems), SCADA network vulnerabilities, and Cyber-Physical Attack Detection mechanisms within modern power grids.